

Reflection

RelayPay Support Agent · Week 6 · Emmanuel Aboyeji · 2026-10-03

In a business setting, what clarifying questions would you ask?

The brief says "customer support agent", which hides most of the decisions.

1. ****What is the agent allowed to *do*, not just say?*** Reading a transaction status and issuing a refund are different products with different risk. I built read-only lookups plus ticket and escalation creation, and deliberately gave it no ability to move money or send email.
2. **What does failure cost?** For a payments company, a confident wrong answer about someone's money is worse than no answer. That single assumption drove the whole design: everything substantive comes from a tool, and anything ungrounded is declined and routed to a human.
3. **Who is liable for what the agent says?** This decides whether the agent may paraphrase policy or must deliver it verbatim, and whether calls need a recording disclosure. I assumed yes and put it in the greeting.
4. **What is the real call volume and the latency budget?** A voice line that takes 25 seconds to answer is unusable even if every answer is right. This turned out to be the dominant engineering constraint.
5. **Where does the knowledge base actually live, and who keeps it current?** An agent grounded in a stale KB is a confident liar with extra steps.
6. **What happens at the handoff?** Is there a human queue to escalate into, and what do they need to see? That shaped the admin console and the audit trail.

The most significant challenge, and its root cause

Latency — and the root cause was architectural, not a slow model.

Turns took 34–68 seconds, and a turn that called no tools at all still took 39 seconds. That last measurement is what cracked it: if an empty turn costs 39 seconds, the model and the database are not the problem.

The Claude Agent SDK runs the agent in a **Claude Code subprocess**, and `query()` spawns a fresh one per call. I was paying full process startup on every single conversational turn.

The fix was to stop treating a turn as a process. In streaming-input mode the SDK takes an async iterable as its prompt, so one subprocess serves a whole call: turns are pushed onto a queue and the process parks between them. Then the spawn is paid once per call instead of once per turn — and it can be paid *while the caller is listening to the greeting*, so it costs nothing.

	BEFORE	AFTER
First question	25–44s	13.7s
Follow-up turns	34–68s	7–9s

Two things made this harder than it sounds. The subprocess emits nothing until the first user message, so there is no "ready" event to wait for — warming is simply calling `query()` early and not awaiting anything.

And a persistent session fixes its system prompt at spawn time, before anyone knows whether the call will escalate, so the state-dependent half of the prompt had to move into each turn.

One thing I would do differently

I would treat the deployment target as a design input on day one, not a packaging step at the end.

I built against Vercel because that is the default reflex for a Next.js app. It cannot work. The Agent SDK ships its CLI as a ~238MB per-platform native binary, and a Vercel serverless function is capped at 250MB unzipped — before Next and the rest of `node_modules`. *The smaller bundled `cli.js` does fit, but MCP tools never attach to it; the agent loops on tool discovery and never finds `Lookup` `transaction`.*

Both facts are discoverable in an afternoon. Instead I found them after the app was built, through a deploy that *succeeded* and then failed on every turn with a message pointing nowhere. Moving to Render solved it and was better anyway — a persistent process keeps the subprocess warm between calls, which is exactly what the latency work needed.

The transferable lesson: **"does the runtime fit the host" is a question to answer before writing code, whenever the stack spawns processes or ships native binaries.** A second, smaller one: I would have added the `x-relaypay-degraded` diagnostic header on day one. The moment it existed it told me the real error immediately, where before I had spent an hour guessing at a generic "I'm having trouble" message.

Edge cases accounted for

- **The caller reads out a card number or an OTP.** No prompt can prevent this — by the time text exists, the value is captured. Redaction runs at the *earliest* point, before the model, the tools or any log line. Luhn-checked PANs, spoken digit runs, contextual CVV/OTP/password cues. Ticket and transaction references are protected from false positives. 46 tests.
 - **The model inventing an identifier.** It did exactly this, filing a real ticket under a conversation that never existed. The conversation id is now supplied by the server and overwritten at the tool boundary; a mismatch is logged, because it means the model is still trying.
 - **Prompt injection and claimed authority.** No assertion of being staff, of urgency, or of "this is a test" unlocks a rule. Verified by adversarial probes, not by hoping.
 - **A tool failing mid-call.** Every tool returns a structured result rather than throwing; a database call has a hard timeout and one retry; a dead turn speaks a graceful fallback instead of leaving silence. The route never returns a 500 to Vapi, because that makes the assistant go mute.
 - **The retrieval gap.** Without an embedding key, retrieval falls back to Postgres full-text. That fallback originally found nothing for real questions, because AND semantics required a whole question in one chunk, and words absent from the corpus dragged good matches below the threshold.
 - **A caller pausing mid-sentence.** Vapi emits a transcript event per sentence, so the on-screen record fragmented into a card per clause. Fragments from one speaker now coalesce, with a pause boundary.
 - **Misconfiguration.** Auth fails closed — an unset secret denies everyone rather than admitting everyone. A missing MCP build is reported loudly instead of dying as an unexplained exit code.
 - **Unanswerable questions.** Declined and redirected, never answered from general knowledge.
-

Which Claude model, and why

Claude Haiku 4.5, with **Claude Sonnet 4.5** as the alternative I measured against.

On a voice line, latency *is* correctness — a caller does not experience a slow right answer as a right answer. Measured on a warmed session, same questions, same correct answers drawn from the database:

	TURN 1	TURN 2	TURN 3	WARM AVERAGE
Sonnet 4.5	24.2s	9.8s	19.4s	14.6s
Haiku 4.5	16.7s	7.4s	11.0s	9.2s

A 37% cut in the thing the user actually feels, at materially lower cost per call.

Task complexity is what made this safe. The hard reasoning is not in the model. Retrieval, lookups, deduplication, escalation rules and verdict logic all live in the MCP server and the system prompt; the model routes to a tool and speaks the result in plain language. That is well within Haiku's range. Had the model been responsible for the judgement — deciding whether a refund is warranted — I would have kept Sonnet.

I did not decide on speed alone. Downgrading the model that enforces the security rules needed evidence, so I wrote four adversarial probes — prompt extraction, tool disclosure, balance invention, authority bypass. Haiku held all four. `CLAUDE_MODEL` overrides the default, so reverting is one environment variable.

